NutriCoach Scaling Checklist

From MVP to 1M Users

Phase 1: Foundation (0-10K users)

Infrastructure

- [ ] Single Next.js instance on Abacus.AI hosting
- [ ] Managed PostgreSQL with connection pooling
- [ ] S3 for file storage (share cards, uploads)
- [ ] Basic CloudFlare CDN for static assets

Code Quality

- [ ] Implement structured logging (pino)
- [ ] Add request ID tracking
- [ ] Set up error boundaries in React
- [ ] Add API response time logging

Database

- [ ] Optimize existing indexes
- [ ] Add `EXPLAIN ANALYZE` to slow queries
- [ ] Set up PgBouncer for connection pooling
- [ ] Implement soft deletes where appropriate

Monitoring

- [ ] Health check endpoint (`/api/health`)
- [ ] Basic uptime monitoring (UptimeRobot/Pingdom)
- [ ] Error tracking (Sentry or similar)

Security

- [ ] Rate limiting on auth endpoints
- [ ] Input validation on all API routes
- [ ] CSRF protection
- [ ] Security headers (CSP, HSTS)

Phase 2: Growth (10K-100K users)

Infrastructure

- [ ] Add Redis (ElastiCache) for:
- [ ] Session caching
- [ ] Rate limiting

- [ ] Food catalog caching
- [ ] Calculation results caching
- [ ] PostgreSQL read replica for:
- [ ] Analytics queries
- [ ] Leaderboard reads
- [ ] Report generation
- [ ] Job queue (BullMQ) for:
- [ ] Meal plan generation
- [ ] Share card rendering
- [ ] Email sending
- [ ] Embedding generation

Performance

- [ ] Implement ISR for public pages
- [ ] Cache food catalog in Redis (1hr TTL)
- [ ] Cache user calculations (5min TTL)
- [ ] Implement leaderboard snapshots (hourly)

Database Schema

- [ ] Add `LeaderboardSnapshot` model
- [ ] Add `JobQueue` model
- [ ] Add `AnalyticsEvent` model
- [ ] Add `ShareCard` model
- [ ] Set up table partitioning for events

Observability

- [ ] Implement OpenTelemetry tracing
- [ ] Set up Prometheus metrics
- [ ] Create Grafana dashboards:
- [ ] API latency (p50, p95, p99)
- [ ] Error rates by endpoint
- [ ] Queue depth
- [ ] Cache hit rates
- [ ] Set up alerts for:
- [ ] P95 latency > 2s
- [ ] Error rate > 1%
- [ ] Queue depth > 100
- [ ] Cache hit rate < 70%

Code Changes

- [ ] Implement snapshot aggregation for leaderboards
- [ ] Move heavy computations to background jobs
- [ ] Add circuit breakers for LLM calls
- [ ] Implement graceful degradation

Phase 3: Scale (100K-500K users)

Infrastructure

- [ ] Multiple app instances behind ALB
- [ ] Redis cluster mode
- [ ] Multiple PostgreSQL read replicas
- [ ] Dedicated worker nodes for jobs
- [ ] CDN for all static assets

Performance

- [ ] Implement connection pooling at scale
- [ ] Add query result caching layer
- [ ] Optimize N+1 queries
- [ ] Implement batch processing for events

Database

- [ ] Review and optimize all indexes
- [ ] Consider partial indexes for common filters
- [ ] Implement query timeouts
- [ ] Archive old data (>12 months)

Architecture

- [ ] Evaluate service extraction needs
- [ ] Consider dedicated RAG infrastructure
- [ ] Implement request coalescing for hot endpoints
- [ ] Add request prioritization

Reliability

- [ ] Multi-AZ deployment
- [ ] Automated failover testing
- [ ] Chaos engineering experiments
- [ ] Disaster recovery drills

Phase 4: Production Scale (500K-1M users)

Infrastructure

- [ ] Multi-region deployment
- [ ] Global CDN with edge caching
- [ ] Database sharding evaluation
- [ ] Dedicated vector store for RAG
- [ ] Event streaming (Kafka/SQS)

Performance

- [ ] Full CQRS implementation
- [ ] Aggressive caching with smart invalidation

- [ ] Database denormalization where beneficial
- [ ] Consider GraphQL for flexible data fetching

Architecture Decisions

- [ ] Extract high-traffic services:
- [ ] Leaderboard service
- [ ] Analytics collector
- [ ] Share card generator
- [ ] Implement API versioning
- [ ] Consider edge computing for calculations

Compliance & Security

- [ ] SOC 2 compliance
- [ ] GDPR data handling
- [ ] PCI compliance (if payments)
- [ ] Security audit
- [ ] Penetration testing

Team & Process

- [ ] On-call rotation
- [ ] Incident response playbooks
- [ ] Load testing automation
- [ ] Feature flag system
- [ ] Canary deployments

Key Metrics to Track

Metric	MVP Target	100K Target	1M Target
P50 Latency	< 300ms	< 200ms	< 100ms
P95 Latency	< 1.5s	< 1s	< 500ms
P99 Latency	< 3s	< 2s	< 1s
Error Rate	< 2%	< 1%	< 0.1%
Uptime	99%	99.5%	99.9%
Cache Hit Rate	60%	80%	90%+
Queue Lag	< 60s	< 30s	< 10s
DB Connections	< 50	< 200	< 500

Cost Estimates

Phase	Users	Monthly Cost	Notes
MVP	0-10K	$50-100	Abacus hosting + S3
Growth	10K-100K	$300-500	+ Redis + Read Replica
Scale	100K-500K	$1.5K-2.5K	+ Multi-instance + Workers
Production	500K-1M	$5K-10K	+ Multi-region + CDN

Quick Reference Commands

```
# Database migrations
cd nextjs_space && yarn prisma db push
cd nextjs_space && yarn prisma generate

# Check database size
yarn prisma db execute --stdin <<< "SELECT
pg_size_pretty(pg_database_size(current_database()));"

# Analyze slow queries
yarn prisma db execute --stdin <<< "SELECT * FROM pg_stat_statements ORDER BY
total_time DESC LIMIT 10;"

# Redis cache stats (when implemented)
redis-cli INFO stats | grep -E 'keyspace|hits|misses'

# Queue monitoring (when BullMQ implemented)
node scripts/queue-stats.js

# Load testing
npx autocannon -c 100 -d 30 https://nutricoach-app.abacusai.app/api/health
```

Emergency Procedures

High Latency

1. Check database connection count
2. Check Redis connectivity
3. Check LLM API status
4. Enable circuit breakers
5. Scale up instances

Database Overload

1. Kill long-running queries

2. Redirect reads to replica
3. Increase connection pool
4. Consider emergency caching

Memory Issues

1. Check for memory leaks
2. Restart affected instances
3. Scale horizontally
4. Review recent deployments

Queue Backup

1. Pause non-critical jobs
2. Scale up workers
3. Clear stuck jobs
4. Investigate root cause